Q1:

(a) support at least 2.

Yes. Every customer contains a sequence also contains its subsequences, so extending a sequence can't increase support. Therefore, no extension of an infrequent sequence can be frequent.

Generate sequences by their total number of items.

Count each matching customer once, even if the sequence matches more than one.

First, find all one-item sequences with support at least 2. Let it be L_1.

Secondly, extend each sequence in L_{k-1} in either way. Add an item not already in this last item set, or append a singleton item set. Use items from L_1 and remove duplicate candidates.

Thirdly, delete each possible single item from a candidate, removing an item set if it becomes empty. Reject the candidate if any resulting subsequence is absent from L_{k-1}. Scan the customer sequences and retain candidates with support at least 2 as L_k.

Repeat until a L_k is empty, and output all L_k for $k \geq 2$. This is complete. Deleting any item from the last itemset of a qualifying sequence, removing that itemset if empty, gives a qualifying shorter sequence that can generate the original. Pruning by the ratio would therefore miss S.

Lv	Retained sequences	support
L_1	⟨{A}⟩, ⟨{D}⟩, ⟨{E}⟩, ⟨{G}⟩	2 each
L_2	⟨{A}, {D}⟩, ⟨{A}, {E}⟩, ⟨{A}, {G}⟩,	2 each
L_3	⟨{D}, {E}⟩	2
L_4	⟨{A}, {D, E}⟩	N/A
	None	

For example, only X contains ⟨{D}, {G}⟩. In Y, D and G occur in the same transaction, so they cannot match two successive itemsets. Its support is 1, so it's removed. The final answer consists of the five sequences in L_2 and L_3.

(b) An adaptation is possible. A ratio below 2 cannot be used to prune all extensions. Generate using support then filter by the ratio.

$$r(S) = \frac{\text{support}_{yes}(S)}{\text{support}_{no}(S)}$$

Both supports in this fraction refers to the same sequence S. Take X as Yes and Y as no.

sequence	Yes	No	ratio
$T = \langle\{A\}, \{D\}\rangle$	1	1	1

T is a subsequence of S. T fails the ratio threshold, but S passes it.

$S = \langle\{A, B\}, \{D\}\rangle$ 1000

First, use the lv-by-lv method in Q1(a), retaining a sequence whenever its support among Yes customers is at least 1.
Second, for each retained sequence, count its Yes and No supports. Add it to the answer only if its ratio is at least 2 and $k \geq 2$. If its no support is 0, use a ratio of ∞ as specified.
Third, a sequence that fails the ratio test must still generate the next lv if its yes support is at least 1. Stop only when no longer sequence satisfies this support condition.
This is complete. Every qualifying sequence has at least 1 yes customer, who also contains all its subsequences. Its total support is automatically at least 1, so 0/0 never occurs.
The answer is exactly all sequences with $k \geq 2$ that occur in X but not in Y. Delete any items from $X = \langle\{A, B, C\}, \{D, E\}\rangle$, remove empty itemsets, and exclude the five common sequences in Q1(a). This specifies the complete answer set.
The six items in X are distinct, giving $2^6 - 1 = 64 - 1 = 63$ nonempty subsequences. Remove the 6 one-item subsequences and the 5 common sequences to obtain 52 answers. Each has yes support 1, no support 0 and ratio ∞.

Q2.

(a) First, set minimum support to 1, build an FP-tree, and run FP-growth to record every nonempty frequent itemset and its support. This avoids discarding lower-support closed item-sets before the ranking is known.

Second, For a suffix item, collect the prefix paths leading to its nodes, using node counts as path weights. These form its conditional pattern base. Build the conditional FP-tree and mine recursively.

Third. add U, compute its support, and remove duplicates. Every zero-support itemset other than U has the zero-support proper superset U, so ~~there is~~ it is not closed. No other 0-support itemsets need to be enumerated.

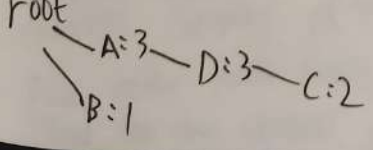
Fourth, for each candidate X, check the entire candidate set. Discard X if a proper superset Y has the same support; otherwise retain it. Step 1 found every positive-support Y, and U suffices for zero support, so this closure check is complete.

Fifthly, sort the retained supports in decreasing order, keeping repeated values. Let τ be the k-th value and output all closed itemsets with support at least τ. Require $1 \leq k \leq$ # closed itemsets. Otherwise the k-th value is undefined.

suffix	conditional pattern base	itemsets and supports
B	empty	B:1
C	⟨A,D⟩:2	C:2, AC:2, CD:2, ACD:2
D	⟨A⟩:3	D:3, AD:3
A	empty	A:3

e.g. C has the conditional tree A:2 → D:2. Combining C with the empty prefix, A, D and AD gives C, AC, CD and ACD, all with support 2.

The transactions are AD, B, ACD, and ACD.
Item supports are A:3, D:3, C:2 and B:1
Order items as A, D, C, B, placing A before D to break the tie.



support	Candidate itemsets	closure check
3	A, D, AD	AD excludes A and D, keep AD
2	C, AC, CD, ACD	ACD excludes C, AC and CD, keep ACD
1	B	No proper superset has equal support, keep B.
0	ABCD (added in step 2)	No proper superset exists. keep ABCD

All non-empty closed itemsets are AD:3, ACD:2, B:1 and ABCD:0. Their sorted support multiset is:[3,2,1,0].

When $k=2$: $\tau=2$, ans = {{A,D}, {A,C,D}}
The second greatest support is 2, so retain the closed itemsets with support 3 or 2.

When $k=3$: $\tau=1$, ans = {{A,D}, {A,C,D}, {B}}
The third greatest support is 1, so retain these three positive-support closed itemsets. ABCD has support 0 and is excluded. It would also be returned for $k=4$.

Compare to frequent itemsets, the advantage is that at the same support threshold, closed itemsets reduce redundant information. At minimum support 1, the 8 non-empty frequent itemsets here are represented by 3 closed itemsets. They preserve frequent-itemset support information. The support of a frequent itemset X equals the largest support among closed itemsets containing X. e.g. AD recovers the support of A as 3. For positive support X, intersect all transactions containing X to obtain a closed superset with equal support. No superset can have greater support. For 0 support, the closed superset U recovers support 0. The disadvantages are ~~that~~ closure checks add work, listing all frequent itemsets or all association rules requires expanding the compressed information. There may still be many closed itemsets in the worst case. With k-based ranking, the support threshold also depends on the data and k.

KAISA

Q3:

(a) In each round, assign each point to its nearest centre, then average the coordinates in each cluster. Stop when assignments no longer change. Squared distances give the same nearest-centre assignments as Euclidean distances.

$d^2((a,b),(c,d)) = (a-c)^2 + (b-d)^2$

$\mu_c = \frac{\sum_{x \in C} x}{|C|}$

first init:

$\mu_1 = (10,1)$, $\mu_2 = (10,30)$

cluster	round 1 and final mbrs:	updated mean:
C_1	$\{1,2,3,4,5,6,7\}$	$(\frac{65}{7}, \frac{67}{7})$
C_2	$\{8\}$	$(10,30)$

e.g. C_1 has coordinate sum $(65,67)$ and 7 pts, so its mean is $(\frac{65}{7}, \frac{67}{7})$. round 2 changes no assignments, so stop.

second init:

$\mu_1 = (3,10)$, $\mu_2 = (15,10)$

round	cluster	mbrs	updated means
1	C_1	$\{2,4,6\}$	$(\frac{7}{3}, \frac{31}{3})$
1	C_2	$\{1,3,5,7,8\}$	$(\frac{68}{5}, \frac{66}{5})$
2	C_1	$\{2,4,6,7\}$	$(\frac{17}{4}, 8)$
2	C_2	$\{1,3,5,8\}$	$(\frac{29}{2}, \frac{65}{4})$

In round 2, x_7 moves to C_1 cuz its squared distances to the two updated centres satisfy $d^2(x_7, \mu_1) = \frac{1313}{9} < \frac{809}{5} = d^2(x_7, \mu_2)$

No other point moves, and round 3 changes no assignments. Thus the two clusters and means from round 2 are the final answer.

three init centres:

$\mu_1 = (10,1)$ $\mu_2 = (10,30)$ $\mu_3 = (3,10)$

cluster	round 1 and final mbrs	updated means
C_1	$\{1,3,5,7\}$	$(\frac{29}{2}, 9)$
C_2	$\{8\}$	$(10,30)$
C_3	$\{2,4,6\}$	$(\frac{7}{3}, \frac{31}{3})$

Round 2 changes no assignments, so there are the final clusters and means.

Four init centres:

$\mu_1 = (10,1)$, $\mu_2 = (10,30)$, $\mu_3 = (3,10)$, $\mu_4 = (15,10)$

cluster	round 1 and final mbrs	updated mean
C_1	$\{7\}$	$(10,1)$
C_2	$\{8\}$	$(10,30)$
C_3	$\{2,4,6\}$	$(\frac{7}{3}, \frac{31}{3})$
C_4	$\{1,3,5\}$	$(16, \frac{35}{3})$

Round 2 changes no assignments, so these are the final clusters and means.

pro: the method is simple and its centres are easy to interpret. Each round takes about $O(nkd)$ basic ops, making it useful for large numerical datasets. Standard assignment and mean updates do not increase squared error, but do not guarantee a global optimum. Here n is the number of points and d is the dimensions.

cons: limitation — improvement or alternative:

① k must be a priori specified. → Compare silhouette scores.

② sensitive to initial centres → Use k-means++, which chooses the first centre uniformly from the data points, then sample each next centre with probability proportional to $D(x)^2$, where $D(x)$ is the distance to the nearest chosen centre, and run multiple times. Keep the smallest squared error.

③ outliers can pull the mean. → check anomalies or use k-medoids.

④ poor fit curved, non-convex clusters. → Use other methods like linkage or DBSCAN

⑤ Distances depend on scale, means do not suit categories → Standardise numerical features when it is appropriate. Use k-modes for categories.

KAIST

Q4:

Start with one point per cluster.
At each step, merge the two clusters with the smallest average distance, defined as the mean over all pairs with one point in each cluster.

$$D(A,B) = \frac{\sum_{x\in A}\sum_{y\in B} d(x,y)}{|A|\,|B|}$$

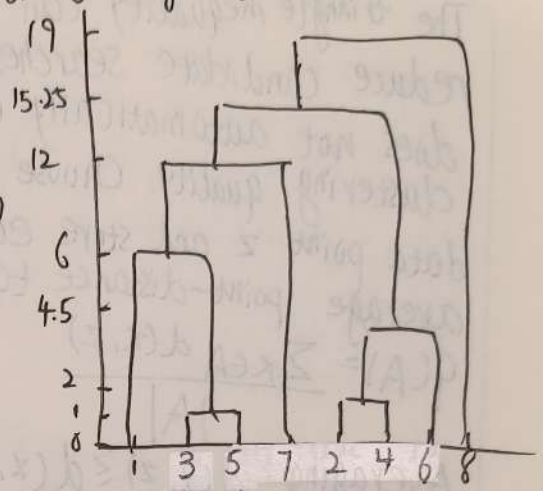
After a merge, update distances using cluster sizes as weights.

$$D(A\cup B, C) = \frac{|A|\,D(A,C) + |B|\,D(B,C)}{|A|+|B|}$$

step	clusters merged	merge distance
1	{3}+{5}	1
2	{2}+{4}	2
3	{2,4}+{6}	$\frac{4+5}{2}=4.5$
4	{1}+{3,5}	$\frac{5+7}{2}=6$
5	{1,3,5}+{7}	$\frac{9+12+15}{3}=12$
6	{1,3,5,7}+{2,4,6}	$\frac{183}{12}=15.25$
7	{1,2,3,4,5,6,7}+{8}	$\frac{133}{7}=19$

Step 6 averages $4\times 3 = 12$ cross-cluster pairs. Grouping by points 1, 3, 5 and 7 gives distance sums 36, 42, 55 and 50, totaling 183. In the last step, distances to point 8 sum to 133.

average-linkage distance.



The vertical axis is the average-linkage distance defined above, using the given point-distance matrix. The left-to-right leaf order does not affect the clustering.

Starting with 8 singleton clusters, the first 4 merges leave these 4 clusters: {1,3,5}, {2,4,6}, {7}, {8}

Cutting the dendrogram at any height strictly between 6 and 12 gives this partition. Moving up this tree, the first cluster containing both points is {1,3,5}. Later steps can only merge it, not split it.

{1,3,5} → {1,3,5,7} → {1,2,3,4,5,6,7} → {1,2,3,4,5,6,7,8}

Therefore, the maximum is 8 pts and the minimum is 3 pts.

The triangle inequality can help reduce candidate searches, but does not automatically improve clustering quality. Choose a fixed data point z and store each cluster's average point-distance to z:

$$q(A) = \frac{\sum_{x \in A} d(x, z)}{|A|}$$

Averaging $d(x, z) \le d(x, y) + d(y, z)$ over $x \in A$ and $y \in B$ gives $q(A) \le D(A, B) + q(B)$. Swap A and B to obtain:

$$D(A, B) \ge |q(A) - q(B)|$$

This bound needs only a subtraction and an absolute value. If a pair of current clusters has known distance b, exclude pairs with $|q(A) - q(B)| > b$. With clusters sorted by q, search for A only within $q(B) \in [q(A) - b, q(A) + b]$. Whole ranges outside it can be skipped. After a merge, the stored average can be updated directly

$$q(A \cup B) = \frac{|A|\, q(A) + |B|\, q(B)}{|A| + |B|}$$

Actual speed depends on how many candidates are excluded and the cost of maintaining the order. If all cached cluster distances are still scanned pair by pair, the extra bound check may instead slow the method down. Under the same tie-breaking rule, such exact pruning preserves the clustering result.

Q5:

The rule does not guarantee a good k. Finer partitions can keep reducing training error. However, almost no further reduction does not guarantee to identify a meaningful number of clusters.

One counterexample is that the 4 1d pts $\{0, 1, 10, 11\}$ form 2 well-separated grps. Even if every run finds a globally optimal k-means solution.

k	one optimal set of centres	sum of distances e_k	decrease
1	5.5	20	∞
2	0.5, 10.5	2	18
3	0, 1, 10.5	1	1
4	0, 1, 10, 11	0	1

e.g. $e_1 = 5.5 + 4.5 + 4.5 + 5.5 = 20$ and $e_2 = 0.5 + 0.5 + 0.5 + 0.5 = 2$. If the small-number threashold is below 1, the rule stops neither at $k=2$ nor at $k=4$. And, next attempts $k=5$, exceeding the number of points. Also, $e_0 = \infty$ prevents the rule from returning $k=1$. The stated e_k is a sum of distances, whereas standard k-means optimises a sum of squared distances.

Compare candidate values of k using the average silhouette score. It measures both within-cluster closeness and between-cluster separation. Below, assume $n \geq 3$ and at least two distinct data locations.

First, let m be the # distinct data locations. Try $k = 2, \ldots, \min(n-1, m)$, with at most R initializations and T rounds per run, fixed in advance. For each k, keep the converged, non-empty cluster run with the smallest squared error. Skip k if no run is valid. Report failure if every candidate fails.

Second, for each pt i, let $a(i)$ be its average distance to the

other pts in its own cluster. Compute its average distance to each other cluster. Let $b(i)$ be the smallest of those averages. Then calculate

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad S(k) = \frac{\sum_{i=1,\dots,n} s(i)}{n}$$

Set $s(i)=0$ for a singleton cluster, and also when the denominator is zero. Choose the k with the largest $S(k)$, breaking ties in favour of the smaller k. For the 4-pt example, the optimal clusterings in the table give $S(2)=\frac{359}{399}\approx 0.9$ and $S(3)=\frac{161}{360}\approx 0.447$, so this method selects $k=2$. This is better justified than waiting for error reduction to vanish, but does not guarantee the true cluster count for every dataset. Silhouette cannot evaluate $k=1$. If all pts coincide, use $k=1$. For $n<3$, or when deciding whether there is only one cluster, we need to inspect the data.

One suitable method is single-linkage clustering with a distance threshold. Define the distance between two clusters as the distance of their closest pair of pts:

$$D(A,B) = \min_{x\in A,\, y\in B} \|x-y\|$$

Choose a neighbor hood threshold $\varepsilon > 0$ and proceed as follows.

First, start with each pt in its own cluster.

Second, find the two clusters A and B with the smallest single-linkage distance. If $D(A,B) \le \varepsilon$, merge them and update the distances.

Third, repeat until one cluster remains or every inter-cluster distance exceeds ε.

KAIST

Equivalently, connect pairs of pts whose distance is at most ε and return the connected components. Pts in a round cluster connect through short links. Pts in a curved cluster connect through short links along the curve.

A suitable ε must connect each target cluster internally while creating no edge between different target clusters. Under this condition, the connected components are exactly the target clusters.

If ε is too small, a cluster can split. If it is too large, or noise forms a bridge, two clusters can merge incorrectly. This is the chaining effect. Single linkage can handle the illustrated shapes, but cannot guarantee correct clustering for arbitrary data.

KAIST

放大看我右脚

